Tiling the TER L2 to L1 Merge

Corrected End-to-End Implementation Specification

Measured prototype: up to 15.9x on the complete merge path at L = 16,384

Status
This document replaces the original 8-page implementation note as the implementation authority for the next coding pass. The speed measurements remain prototype measurements unless explicitly labeled repository-integrated. The implementation described below has not yet been claimed as completed.

Scope
Implement one exact cache-tiled Q3REPORT primitive, integrate it first as a count-only comparator, then as a witness reporter in the exact merge layer, and finally in Full TER L2->L1 with the current ternary MITM retained as a correctness fallback. No Bloom filter, parallelism, tile autotuning, L3 streaming, or NTT changes in this change set.

Source basis

- Original document: tiling_implementation_idea.pdf (8 pages).
- Repository snapshot inspected: commit 7e25779b878697bf755c63a76c80e73e83447ef0.
- Relevant repository files: include/ter/q3report_paper_crossover.hpp, src/q3report_paper_crossover.cpp, include/ter/merge.hpp, src/merge.cpp, include/ter/full_ter.hpp, src/full_ter.cpp, include/ter/arithmetic.hpp, include/ter/telemetry.hpp, CMakeLists.txt.
- The prototype results in Section 10 are preserved from the original note. Repository integration and full solve_full_ter speedups remain to be measured.

Contents

1. Executive summary and corrected claims
2. Exact problem and quotient arithmetic
3. Current repository bottlenecks
4. Cache-tiling construction
5. Correctness proof
6. Deterministic parameter policy
7. Reusable tiled Q3REPORT API
8. Count-only kernel
9. Exact witness-reporting kernel
10. Measured prototype results
11. Integration A: paper-crossover comparator
12. Integration B: exact merge.cpp reference and packed paths
13. Integration C: Full TER L2->L1 production path
14. Memory accounting and fail-closed behavior
15. Telemetry and CSV requirements
16. Tests and verification gates
17. Implementation order for Codex / agentic execution
18. Non-goals and deferred work
19. Consequence for dense NTT
20. Definition of done
Appendix A. File-by-file change map
Appendix B. Pseudocode reference
Appendix C. Review checklist

1. Executive summary and corrected claims

The measured result is strong, but the original note mixed a finite benchmark relationship, the count-only comparator, and the actual Full TER implementation. This revision separates them and turns the idea into a precise end-to-end implementation plan.

Claim	Correct statement
What tiling changes	It changes work order and memory locality. The worst-case pair scan remains Theta(|A||B|).
What can be skipped	A nonempty target window visits every pair in its bucket group. A completely empty C-target window may skip that whole bucket group exactly, because no arithmetic witness can exist there.
Finite ladder scaling	The declared benchmark ladder has lambda = |A||B|/Q around 2-4, so Q is numerically close to L^2/lambda at those points.
Paper asymptotic scaling	Using the repository paper exponents, L = 2^(0.2147n) and Q = 2^(0.40385n), so Q is approximately L^1.881, not asymptotically Theta(L^2).
Actual Full TER merge	The real src/full_ter.cpp native ternary merge materializes and sorts all A x B pair records. The nested-loop count-only comparator does not.
Novelty claim	This is not a new tiling concept or a new exponent. It is potentially a new exact cache-local witness-reporting implementation architecture for TER.

Core implementation objective
Replace cache-hostile Q-wide lookup and, in production Full TER, the L^2 pair-record materialization with one exact tiled reporter that naturally returns (a,b,c) witnesses while reusing the repository's existing structural validation and finalization logic.

2. Exact problem and quotient arithmetic

For one L2->L1 ternary merge, the three child lists A, B, C contain entries whose residues are fixed modulo m2 = 2^r2 by their child targets. The parent is constrained modulo m1 = 2^r1. Let Q = m1/m2 = 2^(r1-r2).

```
q_x = ((residue_m1(x) - child_target_x) / m2) mod Q
q_a + q_b + q_c == tau (mod Q)
```

The repository already represents this arithmetic in MergeArithmetic and exposes quotient_residue(...). The tiled algorithm must consume those existing semantics rather than invent a second definition of q or tau.

For the symmetric three-list arithmetic equation, any pair orientation is valid. The exact merge layer should continue to use choose_pair_orientation(...) so the two smallest lists can be paired when list sizes differ.

3. Current repository bottlenecks

There are three distinct existing paths, and the implementation must improve them without confusing their roles.

Path	Current behavior	Problem
q3report_paper_crossover / DIRECT_PRODUCTION	Builds a Q+1 offset table over C and scans every A x B pair with a random lookup into that table.	Count-only, O(Q) index, cache-hostile at large Q.
src/merge.cpp pair baseline	Builds Q+1 offsets, then materializes an L^2 packed pair array keyed by quotient residue before recovery/validation.	O(Q + L^2) auxiliary search storage.
src/full_ter.cpp ternary_mitm_native	Materializes PairRecord64 for every A x B pair, sorts them, then binary-searches by each C entry.	O(L^2) pair storage plus sorting; this is the production bottleneck architecture to remove for L2->L1.

The tiled reporter is therefore more than a faster benchmark lookup. In production it can also remove the pair-record array and pair sort while preserving exact witness identities.

4. Cache-tiling construction

Choose a power-of-two tile width S. Partition [0,Q) into P = Q/S buckets. For an A entry in bucket i and a B entry in bucket j:

```
q_a in [iS, (i+1)S)
q_b in [jS, (j+1)S)
=> q_a + q_b in [(i+j)S, (i+j+2)S)
```

The sum window depends only on m = i+j. Therefore all bucket pairs with the same m can be processed while a single 2S-wide exact C-target structure is hot in cache.

4.1 Re-key and unwrap C

For each z in the lookup list, compute k_z = (tau - q_z) mod Q. Because 0 <= q_a+q_b < 2Q, modular equality is equivalent to ordinary integer equality against exactly one of k_z or k_z+Q. Store both records.

```
D = sorted records {
    (key = k_z,     z_id),
    (key = k_z + Q, z_id)
    for every z
}
```

Use 64-bit keys and window bounds even though quotient residues are 32-bit. The doubled key can approach 2Q-1, and the implementation should not depend on accidental 32-bit wraparound.

4.2 Bucket A and B without sorting pair sums

Counting-sort A and B by bucket index q >> tile_bits. Preserve each original EntryId. Do not sort or materialize A x B pair sums. The bucket order induces locality for the pair scan.

4.3 Overlapping windows and selective clearing

Successive windows [mS,(m+2)S) overlap by S. A doubled C record can therefore appear in at most two consecutive windows. Keep a monotone lower cursor lo, recompute hi from lo for each window, scatter only the records in that window, and clear only touched slots afterward. The total C scatter/clear work stays O(|C|) up to a constant factor.

5. Correctness proof

The optimized path is exact. Its correctness does not depend on random quotient distribution, cache size, Bloom-filter behavior, or a heuristic selectivity assumption.

- Pair partition: every pair (a,b) has unique bucket indices i=floor(q_a/S) and j=floor(q_b/S), hence a unique group m=i+j. Therefore no pair is assigned to two groups.
- Window containment: for that unique group, q_a+q_b lies in [mS,(m+2)S), so its exact target slot is inside the active window.
- Modulo unwrapping: if q_a+q_b+q_c == tau (mod Q), then q_a+q_b is exactly k_c or k_c+Q because the pair sum lies in [0,2Q).
- Multiplicity preservation: every C entry creates its own doubled records. Duplicate q_c values therefore remain distinct witnesses and are not deduplicated in the arithmetic reporter.
- Empty-window skip: if D contains no target record in the active window, no pair in that group can satisfy the arithmetic equation, so skipping the group is exact.
- Structural semantics unchanged: arithmetic hits are passed to the existing audit/packed validators and existing finalization logic. Tiling must not reimplement ternary structural validity or parent deduplication.

6. Deterministic parameter policy

The first repository implementation must use one deterministic policy so benchmark results are attributable to the algorithm rather than an autotuner.

```
q_bits   = log2(Q)              // Q must be a power of two
SB        = min(15, q_bits)
S         = 1ULL << SB
P         = Q / S
window    = 2*S slots
```

Case	Behavior
q_bits < 15	S = Q and P = 1. The algorithm degenerates to one window and remains correct.
q_bits = 15	S = Q; one 2Q-wide unwrapped window.
q_bits > 15	S = 32768; the count table/head table has 65536 slots.
Unsupported Q	If Q is zero, not a power of two, or cannot be represented by the shared quotient API, return NotApplicable/InvalidConfiguration at the experiment layer or use the existing fallback at the solver layer.

No autotuning in V1
Do not sweep S in production logic. Tests may run several explicit tile sizes to verify invariance. A performance sweep is deferred until fixed-S correctness is locked down.

7. Reusable tiled Q3REPORT API

Create one reusable component rather than three copies of the same loop.

File	Responsibility
include/ter/tiled_q3report.hpp	Public exact count/report types and entry points.
src/tiled_q3report.cpp	Bucket build, doubled C-key build, count kernel, witness kernel, memory preflight, telemetry.
tests/tiled_q3report_tests.cpp	Standalone arithmetic correctness tests independent of Full TER.

```
struct TiledQEntry {
  QuotientResidue q = 0;
  EntryId id = 0;
};

struct TiledQ3ReportConfig {
  std::uint32_t preferred_tile_bits = 15;
  std::uint64_t memory_limit_bytes = 0;
};

struct TiledQ3ReportTelemetry {
  std::uint32_t tile_bits = 0;
  std::uint64_t tile_width = 0;
  std::uint64_t window_width = 0;
  std::uint64_t bucket_count = 0;
  std::uint64_t window_count = 0;
  std::uint64_t logical_pairs = 0;
  std::uint64_t executed_pairs = 0;
  std::uint64_t raw_triples = 0;
  std::uint64_t nonempty_windows = 0;
  std::uint64_t doubled_lookup_records = 0;
  std::uint64_t auxiliary_peak_bytes = 0;
  std::uint64_t setup_ns = 0;
  std::uint64_t scan_ns = 0;
};

enum class TiledVisitDecision : std::uint8_t {
  Continue,
  ResourceLimit,
  Cancelled,
  InvariantViolation,
};

using TiledTripleVisitor = TiledVisitDecision (*)(
    void* context, EntryId x, EntryId y, EntryId z);
```

Expose one count-only entry point and one visitor entry point, or one shared entry point with a null visitor. Internally share bucket/key preparation and keep the hot loops specialized so the count-only benchmark does not pay callback overhead.

8. Count-only kernel

Count-only mode is the direct replacement candidate for the paper-crossover comparator, not for production witness recovery.

```
allocate count[0 .. 2S) as uint32_t
build bucketed A and B
build sorted doubled C records D
lo = 0
raw = 0

for m = 0 .. 2P-2:
    w0 = m*S
    w1 = w0 + 2S

    while lo < |D| and D[lo].key < w0:
        ++lo
    hi = lo
    while hi < |D| and D[hi].key < w1:
        ++count[D[hi].key - w0]
        ++hi

    if lo != hi:
        for every valid i with j = m-i and 0 <= i,j < P:
            for x in A_bucket[i]:
```

```
                for y in B_bucket[j]:
                    sum = uint64(x.q) + uint64(y.q)
                    assert(w0 <= sum and sum < w1)
                    raw += count[sum - w0]
                    ++executed_pairs

    for t = lo .. hi-1:
        count[D[t].key - w0] = 0
```

The count array is only 2S entries. At SB=15 and uint32_t counts this is 256 KiB. If a slot multiplicity could exceed UINT32_MAX in a future regime, the method must fail closed or widen the counter type; do not silently overflow.

9. Exact witness-reporting kernel

Production merge integration needs C identities, not merely counts. Use a cache-resident per-slot linked list over the doubled C records.

```
NONE = UINT32_MAX
head[0 .. 2S) = NONE
next[0 .. |D|) = NONE

for each active window:
    for record = lo .. hi-1:
        slot = D[record].key - w0
        next[record] = head[slot]
        head[slot] = record

    for each pair (x,y) in the bucket group:
        slot = (uint64(x.q) + y.q) - w0
        record = head[slot]
        while record != NONE:
            ++raw_triples
            visitor(context, x.id, y.id, D[record].z_id)
            record = next[record]

    for record = lo .. hi-1:
        head[D[record].key - w0] = NONE
```

Because EntryId is already 32-bit in the repository, a uint32_t record chain is acceptable only when 2|C| <= UINT32_MAX. Preflight that condition. If it fails, the optimized path is unsupported and the solver must fall back rather than truncate.

One pass, not two
Do not first count all modular witnesses and then rerun the L^2 scan for recovery. That would sacrifice most of the measured win. Production validation must happen during the single witness-reporting pass, with conservative incremental memory accounting for accepted candidates.

10. Measured prototype results

These tables are preserved from the original prototype note. They are measurements of standalone programs linked against the repository library, not yet results from an integrated repository implementation.

q_bits	L	lambda	current	tiled	speedup	reported peak memory
23	4,096	2.0	81.7 ms	6.9 ms	11.9x	32 MiB -> 0.3 MiB
24	8,192	4.0	373.5 ms	26.3 ms	14.2x	64 MiB -> 0.4 MiB
26	16,384	4.0	1951.4 ms	149.8 ms	13.0x	256 MiB -> 0.6 MiB

L	Q	arith current	arith tiled	structural	dedup	merge current	merge tiled	speedup
2,048	2^21	9.1 ms	1.5 ms	0.7 ms	0.3 ms	10.1 ms	2.5 ms	4.08x
4,096	2^23	92.8 ms	5.8 ms	1.4 ms	0.6 ms	94.8 ms	7.8 ms	12.10x

L	Q	arith current	arith tiled	structural	dedup	merge current	merge tiled	speedup
8,192	2^24	398.4 ms	22.1 ms	6.7 ms	2.3 ms	407.4 ms	31.0 ms	13.13x
16,384	2^26	2199.8 ms	120.0 ms	14.5 ms	5.1 ms	2219.4 ms	139.6 ms	15.90x

At L=16,384 the original note reports 65,524 arithmetic witnesses, 22,528 structurally valid triples, and 22,485 unique parents. Those figures must be reproduced by the integrated exact path before any production selector is changed.

11. Integration A: paper-crossover comparator

The historical DIRECT_PRODUCTION comparator is part of a completed experiment and must remain untouched. Add a separately named tiled method and append telemetry/CSV fields.

- Add direct_production_tiled_method(...) in src/q3report_paper_crossover.cpp. It must call the shared count-only tiled primitive.
- Add PaperMethodOutcome direct_tiled to PaperCrossoverTelemetry.
- Add direct_tiled_setup and direct_tiled_scan timing summaries to PaperComponentTimings.
- Add tile metadata: tile_bits, window bytes, nonempty windows, logical pairs, executed pairs, and auxiliary peak bytes.
- In untimed Phase A, assert exact raw count equality across direct_full_pair_counts, existing DIRECT_PRODUCTION, DIRECT_PRODUCTION_TILED, and the selected-target sum recovered from NTT coefficients whenever the NTT path is applicable.
- In timed Phase B/C, benchmark the new method independently with the same warm-up/repetition policy. Do not reuse another method's setup allocation or timing state.
- Append CSV columns; never rename, reorder, or overwrite existing historical columns.

```
Suggested appended CSV fields:
  direct_tiled_total_median_ns
  direct_tiled_setup_median_ns
  direct_tiled_scan_median_ns
  direct_tiled_peak_bytes
  direct_tiled_tile_bits
  direct_tiled_nonempty_windows
  direct_tiled_executed_pairs
  ntt_over_direct_tiled
```

12. Integration B: exact merge.cpp reference and packed paths

This is the first true witness-reporting integration. It should change only arithmetic candidate location. Existing validators and output canonicalization remain the correctness authority.

```
// include/ter/merge.hpp
MergeOutput run_pair_tiled(const InputSnapshot&, OutputPolicy);
MergeOutput run_pair_tiled_packed(const PackedSnapshot&, OutputPolicy);
```

- Audit the input snapshot exactly as the existing pair baseline does.
- Use choose_pair_orientation(...) for the reference path and the same three-choice minimum-pair-cost rule for the packed path.
- Build TiledQEntry arrays from the selected x/y/z roles, preserving entry.id and entry.q.
- Run the witness reporter with tau = snapshot.spec.arithmetic.tau and Q = snapshot.spec.arithmetic.q.
- For every reported x_id,y_id,z_id, map IDs back into canonical A/B/C role order before validation.
- Reference path calls the existing validate_and_collect/audit_candidate logic. Packed path calls the existing packed validation logic. Do not duplicate structural ternary addition rules in tiled_q3report.cpp.
- Finalize with the existing finalize_output/finalize_packed_output behavior so parent multiplicity, provenance sorting, canonicalization, independent M1 audit, and output ordering remain unchanged.

12.1 Incremental memory accounting

The existing pair baseline pre-counts modular candidates so it can reserve worst-case candidate/output buffers. The tiled one-pass reporter should not do a second L^2 count pass. Instead, preflight fixed auxiliary memory, derive a conservative maximum number of accepted candidates from the remaining memory budget, and fail before an append would exceed it.

- Reference candidate budget should conservatively include AcceptedCandidate object storage, n digit bytes, eventual ParentRecord/digit storage, and provenance storage when OutputPolicy::Correctness is active.
- Packed candidate budget should include PackedCandidateRecord, 2*words_per_parent uint64_t mask words, eventual output digits/ParentRecord, provenance when active, and fixed scratch masks.
- raw arithmetic candidate count may be much larger than accepted candidate count and does not itself require storage. Increment telemetry for every arithmetic witness even when validation rejects it.

13. Integration C: Full TER L2->L1 production path

Do not replace generic ternary_mitm globally in the first change. Introduce a specialized native-width L2->L1 tiled path and select it only where quotient arithmetic can be represented exactly by the existing MergeArithmetic API.

```
ternary_tiled_l2_to_l1_native(
    first, second, third,
    child_target_a_r2,
    child_target_b_r2,
    child_target_c_r2,
    parent_target_r1,
    r2, r1,
    expected_level1_counts,
    memory_limit_bytes,
    telemetry,
    provenance,
    status,
    message)
```

At the solve_full_ter L2->L1 call site, the three child targets are targets.level2[group*3 + {0,1,2}] and the parent target is targets.level1[group]. Build MergeArithmetic from the first two child targets and the parent target, then assert that the derived target_c2 matches the stored third child target.

```
ArithmeticResult ar = build_arithmetic(
    n, r2, r1,
    child_target_a_r2,
    child_target_b_r2,
    parent_target_r1);

require(ar.status == Completed);
require(ar.value.target_c2 == child_target_c_r2);
```

For every BigEntry child, mask/convert its residue to the exact r1-width native value and compute q with quotient_residue(...). The optimized path is allowed only when r1 <= 64 and r1-r2 <= 32 so the existing native quotient types are exact. Otherwise call the existing ternary_mitm path.

- Run the tiled witness reporter over the three quotient lists.
- For each arithmetic witness call the existing add_three(...) on the original packed digits with the expected level-1 digit counts.
- On success, construct BigEntry with the same parent digits and same residue expression currently used by ternary_mitm_native.
- If provenance collection is enabled, emit the same ProvenanceRecord fields with the original list indices.
- After the scan, run the same canonicalize_provenance(...) and canonicalize(result) logic. Update survivors_before_dedup/survivors_after_dedup exactly as today.
- Retain current ternary_mitm as the fallback for unsupported widths and as a correctness comparator in bounded audits.

Important scope boundary
Only L2->L1 is authorized in this implementation pass. L1->L0 also uses a ternary merge but has different list sizes and paper-stage economics. Do not silently route it through the new code until separately benchmarked and

audited.

14. Memory accounting and fail-closed behavior

Every optimized path must preserve the repository's research rule: resource uncertainty may stop optimization, but it may not corrupt correctness.

Allocation	Count-only	Witness mode
Bucketed A/B entries	O(|A|+|B|)	O(|A|+|B|)
Bucket offsets	O(P)	O(P)
Doubled C records	2|C| records	2|C| records with z_id
Window structure	2S uint32 counts	2S uint32 heads + 2|C| uint32 next
Candidate/output storage	none	incrementally bounded accepted candidates + normal output/provenance
Pair records	none	none
Q-wide table	none	none

- Use checked_add/checked_multiply-style overflow guards before every size computation.
- If fixed tiled auxiliary memory alone exceeds memory_limit_bytes, return ResourceLimit/NotApplicable as appropriate.
- If accepted output growth would cross the remaining memory budget, stop before appending and return ResourceLimit. Never partially claim Completed.
- Catch std::bad_alloc and std::length_error at the public wrapper boundary, matching existing repository style.
- Fallback in solve_full_ter is allowed only when the tiled path is unsupported before producing output. A mid-run ResourceLimit is a resource outcome, not permission to silently rerun a different algorithm under the same accounting unless the caller explicitly budgets for that fallback.

15. Telemetry and CSV requirements

Telemetry must make the optimization auditable rather than merely fast.

Layer	Required fields
TiledQ3ReportTelemetry	tile_bits, S, 2S, P, window_count, nonempty_windows, logical_pairs, executed_pairs, raw_triples, doubled_lookup_records, auxiliary_peak_bytes, setup_ns, scan_ns.
MergeTelemetry	retain all existing fields; pair_records should be 0 for tiled; pair_combinations remains logical pair universe; candidate_triples equals arithmetic witnesses; add tiled setup/scan/aux fields or embed TiledQ3ReportTelemetry.
FullTerStageTelemetry	add tiled_used, tile_bits, tiled_pair_probes/executed_pairs, tiled_nonempty_windows, tiled_auxiliary_bytes, tiled_setup_ns, tiled_scan_ns without changing existing counters.
Paper CSV	append tiled timing/memory fields and NTT/tiled ratio. Historical direct/NTT fields remain byte-for-byte compatible.

Existing output fingerprints must not change when logical outputs are identical. The algorithm-selection bit itself should not be mixed into the parent/output fingerprint if that fingerprint is intended to represent semantic output rather than implementation path.

16. Tests and verification gates

This implementation is not complete when it compiles. It is complete only after the following gates pass.

Test family	Required checks
Standalone arithmetic	empty lists; singleton lists; duplicate q values; all q identical; q=0; q=Q-1; sums Q-1, Q, 2Q-2; empty windows; multiple explicit S; q_bits below/equal/above 15; random differential vs cubic/direct; every emitted witness satisfies the modular equation.
Partition proof tests	for small Q, mark every A x B pair and assert exactly one m group owns it; assert sum always lies inside that group's 2S window.
Count equality	tiled count == direct_full_pair_counts == existing DIRECT_PRODUCTION for random fixtures; also == selected-target NTT result where applicable.
Witness equality	multiset of (a,b,c) IDs from tiled reporter == scalar cubic witness multiset on tiny bounded fixtures.
merge.cpp reference	run_pair_tiled == run_pair_baseline == run_cubic_oracle on bounded fixtures: parents, multiplicities, valid_provenance, counters where semantically comparable.
packed path	run_pair_tiled_packed == run_pair_tiled after canonicalization and independent parent residue audit.
Full TER capture	old L2->L1 native merge vs tiled L2->L1 on every bounded authentic capture where both fit: exact entries, canonical provenance, modular candidate count, survivors before/after dedup, output fingerprint.
Build hygiene	all existing tests; new tiled tests; strict warnings; Release/native benchmark build; ASan+UBSan; no sanitizer findings.

Production selector gate
Do not make tiled L2->L1 the default path until semantic equality has passed on finite-sanity exhaustive fixtures, randomized snapshots, packed/reference differential tests, and authentic Full TER captures.

17. Implementation order for Codex / agentic execution

Implement in reviewable commits. Each phase must be independently correct before the next one starts.

Phase	Deliverable
Commit 1 - standalone primitive	Add tiled_q3report.hpp/.cpp and standalone tests. Count-only and witness modes only. No repository callers yet.
Commit 2 - paper comparator	Add DIRECT_PRODUCTION_TILED, Phase-A equality, telemetry, benchmark timings, appended CSV fields. Preserve historical comparators.
Commit 3 - exact merge reference	Add run_pair_tiled using existing audit_candidate/finalize_output. Differential tests against pair baseline and cubic oracle.
Commit 4 - exact merge packed	Add run_pair_tiled_packed using existing packed validator/finalizer. Differential tests against reference tiled.
Commit 5 - Full TER specialized path	Add native-width specialized L2->L1 tiled merge plus fallback. Do not touch L1->L0.
Commit 6 - solver audits and benchmarks	Add authentic capture equality tests, telemetry, end-to-end finite-sanity benchmark, and paper-snapshot multi-seed benchmark. Only now decide whether to switch the default L2->L1 selector.

Agent rule: if any correctness comparison fails, stop and debug that phase. Do not paper over mismatch by loosening tests, changing canonicalization, or changing the reference path.

18. Non-goals and deferred work

- No Bloom filter in the tiled V1. Once the exact target structure is cache-resident, Bloom hashing may cost more than the lookup it avoids.
- No threading or SIMD in V1. First reproduce the single-threaded measured mechanism.
- No tile-size autotuner. Fixed SB=min(15,q_bits) first; sweep later.
- No L3 prefix-code streaming in this change. That addresses a separate generation/resource-limit problem.
- No changes to dense NTT implementation or its historical verdict data.
- No claim of a better TER time exponent. The pair scan remains Theta(L^2).
- No automatic use at L1->L0 until separately audited and benchmarked.

19. Consequence for dense NTT

The dense-NTT negative result becomes stronger if the tiled comparator reproduces its prototype performance, because the direct side was previously under-optimized.

At the largest point, the uploaded prototype note used an NTT figure around 11.36 s and a tiled complete-merge figure of 139.6 ms, yielding about an 81x ratio. The latest recorded optimized NTT commit reports roughly 11.216 s warm at L=16,384; against 139.6 ms that is about 80x. Treat the ratio as run-specific, not a universal constant.

Correct asymptotic wording is also important: under the paper exponents Q is approximately L^1.881, so dense NTT is approximately L^1.881 log L while the direct pair scan is L^2. The finite benchmark ladder happens to keep lambda near 2-4, making Q numerically close to L^2/lambda there. Tiling changes finite-machine constants and memory traffic; it does not establish a new exponent.

20. Definition of done

- **[ ]** New shared tiled arithmetic primitive exists and is the only implementation of the tiling math.
- **[ ]** Count-only tiled comparator is separately named and historical comparator/CSV semantics are preserved.
- **[ ]** Exact reference and packed tiled merges report the same parents, multiplicities, and provenance as existing exact baselines.
- **[ ]** Full TER L2->L1 uses tiled reporting only under explicit supported-width conditions and otherwise retains the old exact fallback.
- **[ ]** No L^2 pair-record array or Q-wide table exists in the tiled path.
- **[ ]** All fixed and incremental memory allocations are preflighted/fail-closed.
- **[ ]** All new and existing tests pass in Release and sanitizer builds.
- **[ ]** Multi-seed paper-snapshot benchmark reproduces exact counts and records the new timing/memory columns.
- **[ ]** Authentic Full TER capture differential tests preserve output fingerprints.
- **[ ]** End-to-end solve_full_ter finite-sanity benchmark is recorded separately from the prototype numbers.
- **[ ]** No paper or README claim upgrades prototype measurements to integrated measurements before the integrated benchmark actually runs.

Appendix A. File-by-file change map

File	Change
CMakeLists.txt	Add src/tiled_q3report.cpp to ter_t0; add tests/tiled_q3report_tests.cpp executable/test.
include/ter/tiled_q3report.hpp	New exact tiled arithmetic/reporting API and telemetry.
src/tiled_q3report.cpp	New reusable implementation.
tests/tiled_q3report_tests.cpp	New arithmetic/reporting differential and boundary tests.
include/ter/q3report_paper_crossover.hpp	Append direct_tiled method outcome, component timings, and

File	Change
	tile telemetry fields.
src/q3report_paper_crossover.cpp	Add DIRECT_PRODUCTION_TILED and correctness/timing integration.
bench/ter_q3report_paper_crossover_bench.cpp	Append CSV columns and rows; preserve historical columns.
include/ter/merge.hpp	Declare run_pair_tiled and run_pair_tiled_packed.
src/merge.cpp	Add exact tiled reference/packed wrappers reusing validators/finalizers; incremental memory budget.
include/ter/telemetry.hpp	Add tiled telemetry fields if not embedded from the shared structure.
tests/merge_tests.cpp	Reference tiled differential tests.
tests/packed_tests.cpp	Packed tiled differential tests.
include/ter/full_ter.hpp	Extend FullTerStageTelemetry with tiled fields if selected.
src/full_ter.cpp	Add specialized native-width L2->L1 tiled merge and selector/fallback.
tests/full_ter_tests.cpp	Authentic capture and old-vs-new L2->L1 semantic equality tests.
bench/ter_full_bench.cpp	Record end-to-end finite-sanity/fixture benchmark after integration.

Appendix B. Pseudocode reference

```
function prepare(A, B, C, tau, Q, preferred_bits=15):
    require Q is power of two
    q_bits = log2(Q)
    SB = min(preferred_bits, q_bits)
    S = 1 << SB
    P = Q / S

    Ao, Aoff = counting_bucket(A, SB, P)
    Bo, Boff = counting_bucket(B, SB, P)

    D = []
    for z in C:
        k = (tau - z.q) & (Q - 1)
        D.push({k, z.id})
        D.push({k + Q, z.id})
    sort D by (key, id)
    return SB, S, P, Ao, Aoff, Bo, Boff, D

function scan_windows(prepared, mode, visitor):
    lo = 0
    for m in [0, 2P-2]:
        w0 = m*S
        w1 = w0 + 2S
        while lo < |D| and D[lo].key < w0: lo++
        hi = lo
        while hi < |D| and D[hi].key < w1: hi++
        if lo == hi: continue

        build current-window exact target structure from D[lo:hi]
        for i in [max(0,m-P+1), min(P-1,m)]:
            j = m-i
            for x in A_bucket[i]:
                for y in B_bucket[j]:
                    sum = uint64(x.q) + uint64(y.q)
                    process exact matches at slot sum-w0
        clear only touched target slots
```

Appendix C. Review checklist

- **[ ]** Does the shared primitive preserve duplicate C entries as distinct witnesses?
- **[ ]** Are every doubled key and every w0/w1/sum expression 64-bit?
- **[ ]** Does SB=min(15,q_bits) behave correctly for q_bits < 15?
- **[ ]** Does an empty target window skip its group exactly and update executed_pairs honestly?
- **[ ]** Is the count-only path free of callback/witness overhead?
- **[ ]** Does witness mode avoid a second L^2 scan?

- **[ ]** Are validators/finalizers reused rather than reimplemented?
- **[ ]** Does Full TER derive q and tau through existing arithmetic semantics?
- **[ ]** Is target_c2 asserted against the actual third child target?
- **[ ]** Does unsupported native width fall back before producing any tiled output?
- **[ ]** Are old CSV fields untouched and new ones appended?
- **[ ]** Are prototype and integrated benchmark numbers clearly distinguished?
- **[ ]** Is any asymptotic claim consistent with Q approximately L^1.881 under paper exponents?